

ARTI403 – Lab 1

Basics of Programming with Python

Name: Ali Jafar Altaweel

ID: 2240006456

Section: 7MA2

GitHub: <https://github.com/Ali-developer/Image-Processing/tree/main/Lab1>

In this lab, I set up the Python environment for the image processing course and worked with digital images for the first time. The main objective was to understand how an image is loaded, displayed, stored and represented inside the computer, and to show that an image is nothing more than an array of numbers.

Lab Tasks

Task 1 – Operations on the array

The student will be asked to perform different operations on the array using the NumPy library.

Environment Setup

I installed the libraries needed for the course using `pip`: NumPy, SciPy, OpenCV, scikit-image, Pillow and Matplotlib. The manual asks for Anaconda because it ships Jupyter and Spyder already installed, but Python 3.13 and Jupyter were already available on my machine, so I added the missing libraries directly instead. The first cell of the notebook prints the version of every library to confirm the installation worked.

Loading and Displaying an Image with OpenCV

I loaded the cameraman image with `cv2.imread()` and displayed it with Matplotlib. OpenCV returned a NumPy array directly, with the shape (512, 512, 3) and the type `uint8`, meaning 512 rows, 512 columns and 3 color channels.

An important detail I noticed is that OpenCV reads the color channels in **BGR** order while Matplotlib expects **RGB**. Because the cameraman image is grayscale its three channels are identical, so the difference is invisible here, but for a color image the red and blue would be swapped. I included both versions in the notebook to make the difference clear, using `cv2.cvtColor()` to convert properly.

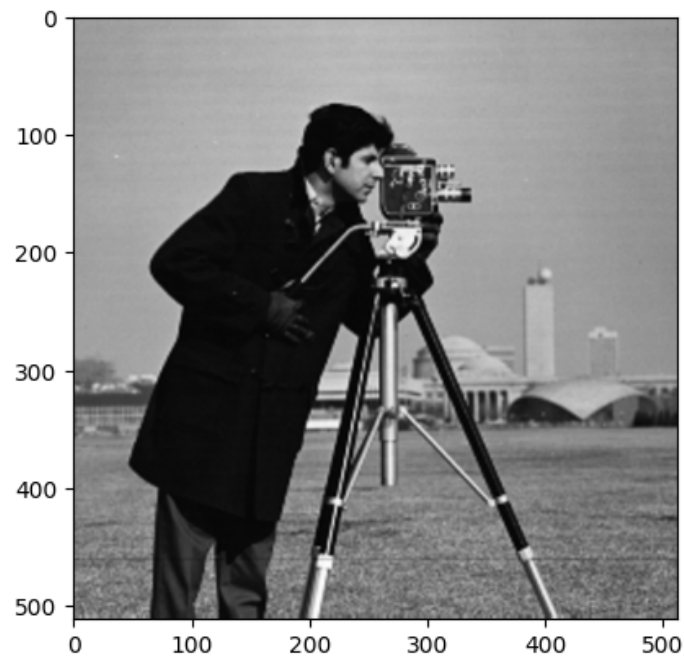


Figure 1 – The cameraman image loaded with OpenCV

cameraman.tif (BGR converted to RGB)



Figure 2 – The same image after converting BGR to RGB

Loading and Displaying an Image with PIL

I then loaded the Lena image with `Image.open()` from Pillow. Unlike OpenCV, this returned a PIL Image object rather than an array. The image had a size of (256, 256) and the mode L, which means 8-bit grayscale. To display it correctly with Matplotlib I used the `Greys_r` colormap.

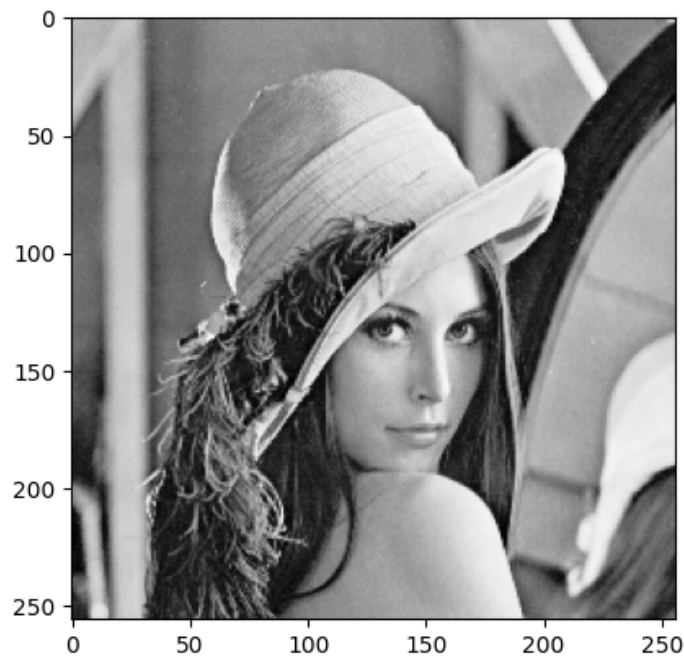


Figure 3 – The Lena image loaded with PIL

Saving an Image to Disk

I saved both images back to disk, using `cv2.imwrite()` for the OpenCV version and `img.save()` for the PIL version, then checked that the two files were really created and printed their sizes.

Both functions decide the output format from the file extension, so reading a `.tif` and writing a `.jpg` performed the format conversion automatically. The file size dropped noticeably after the conversion, because TIFF is lossless while JPEG is a lossy compressed format.

Displaying the Image as an Array

Finally I printed the raw contents of both images, using the array returned by OpenCV directly and converting the PIL image with `np.array()` first. This confirmed that a digital image is stored as a matrix of numbers, where each value is the intensity of one pixel between 0 for black and 255 for white in an 8-bit image.

Operations on the Image Array with NumPy

For the assessment I loaded the cameraman image again, this time with the `IMREAD_GRAYSCALE` flag so it was a single 512×512 array, and applied different NumPy operations to it.

- **Information about the array.** The image holds 262 144 pixels using all 256 gray levels, with a mean of 118.3 and a standard deviation of 62. `argmin()` with `unravel_index()` located the darkest pixel at row 76, column 221.
- **Indexing and slicing.** A slice such as `a[60:260, 160:360]` cropped a 200×200 region around the cameraman, and `a[::4, ::4]` kept every fourth row and column, giving a 128×128 sub-sampled image.
- **Flipping and rotating.** Reversing an axis with `a[::-1, :]` flipped the image upside down and `a[:, ::-1]` mirrored it. I checked that this is exactly the same as `np.flipud()`, and that `np.rot90()` applied four times gives back the original.

- **Arithmetic and logical operations.** `255 - a` produced the negative. Adding 80 to brighten the image made 34 304 pixels wrap around to dark values because of the `uint8` type, so I used `np.clip()` on a wider type instead. The comparison `a > 100` gave a boolean mask covering 71.5% of the image, which `np.where()` turned directly into a black and white image.
- **Reshaping and statistics.** `ravel()` flattened the image into one long vector that reshaped back without any change. The mean of every row and column was computed with the `axis` argument, and `np.bincount()` gave the histogram, where the most frequent gray level was 13, the dark coat of the cameraman.

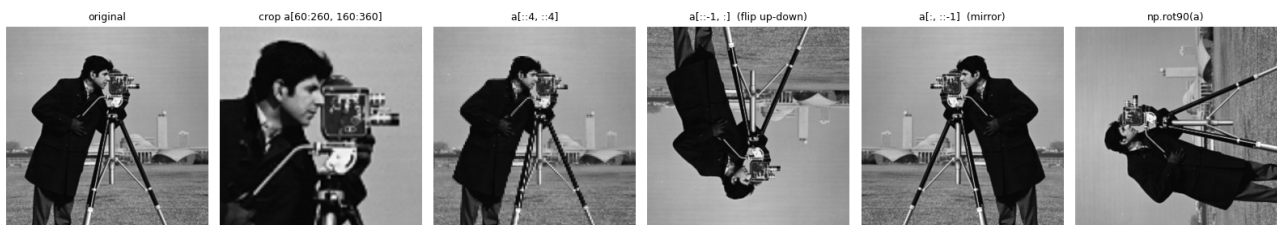


Figure 4 – Task 1: slicing, flipping and rotating with NumPy



Figure 5 – Task 1: arithmetic and logical operations on the array

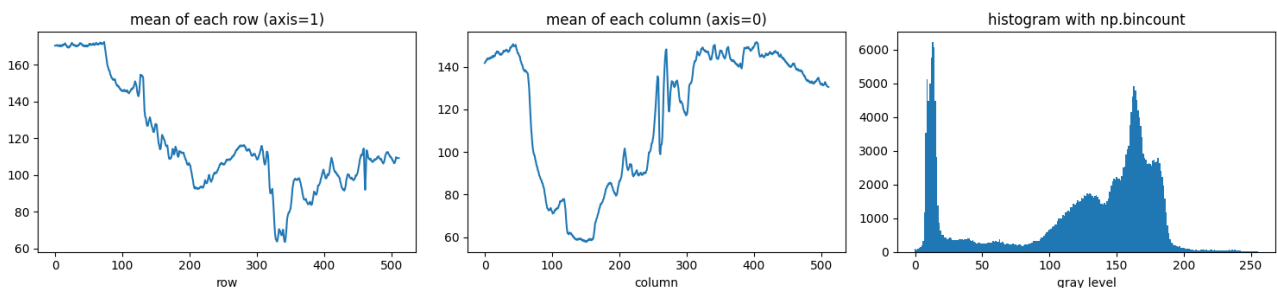


Figure 6 – Task 1: row and column means and the histogram

Observations

The two libraries do the same job but return different things, and this is the main practical difference to remember. OpenCV gives a NumPy array immediately, so it can be processed straight away, but the channels are in BGR order. Pillow gives an Image object that has to be converted with `np.array()` before any numerical work, but it reads the channels in the expected RGB order.

I also noticed that OpenCV loaded the grayscale cameraman image as a three-channel array by default, even though the three channels hold identical values and the file itself is grayscale. Reading it with the `IMREAD_GRAYSCALE` flag returns a single-channel array instead, which is what should be used when the color information is not needed.

Conclusion

This lab showed how the Python environment for image processing is set up and how images are loaded, displayed and stored with the two main libraries of the course.

More importantly, printing the image as an array showed that there is nothing special about the way an image is held in memory. It is a plain NumPy matrix of intensity values, which means that ordinary array operations are already image processing operations.

Submission

The complete work for this lab is in my GitHub repository, in the **Lab1** folder: <https://github.com/Ali-develober/Image-Processing/tree/main/Lab1>

As required, the Lab1 folder contains the following files:

- Jupyter notebook: Lab1.ipynb, executed, with all the outputs and the task solutions
- Images used: images/cameraman.tif, images/lena_gray_256.tif
- This report: Lab1_Report.pdf, and the same write-up as README.md